

Abstract

Computer vision - emergent branch of **artificial intelligence**

- developing algorithms
- pass large amounts of image and video data to computers to help them make meaningful conclusions

Applying computer vision to

- detect **pedestrian trespassing**
- perform **object detection** of camera footage from NJ Transit railroad crossings

Current models: detect trespassing with **over 90% accuracy**,

- near perfect accuracy is crucial to make decisions
- alerting trains to halt because of an obstacle in the tracks
- not enough training data for trespassing

My focus:

- annotating images with labels so that the computer vision model can take those images as input and learn how to better detect pedestrians
- developing a program:
 - **input**: data set of images of people in various scenarios
 - **Microsoft Common Objects in Context (COCO) data set**
 - extract the person
 - **output**: using a customized pipeline of generative models, overlays them on an image of railway tracks to simulate trespassers

Computer vision model learns **new features** and **patterns** that will help it make more **accurate** and **confident decisions**.

Background



Grade crossings: intersection of railroads and roads

- 34% of railroad incidents over the past ten years have occurred at grade crossings

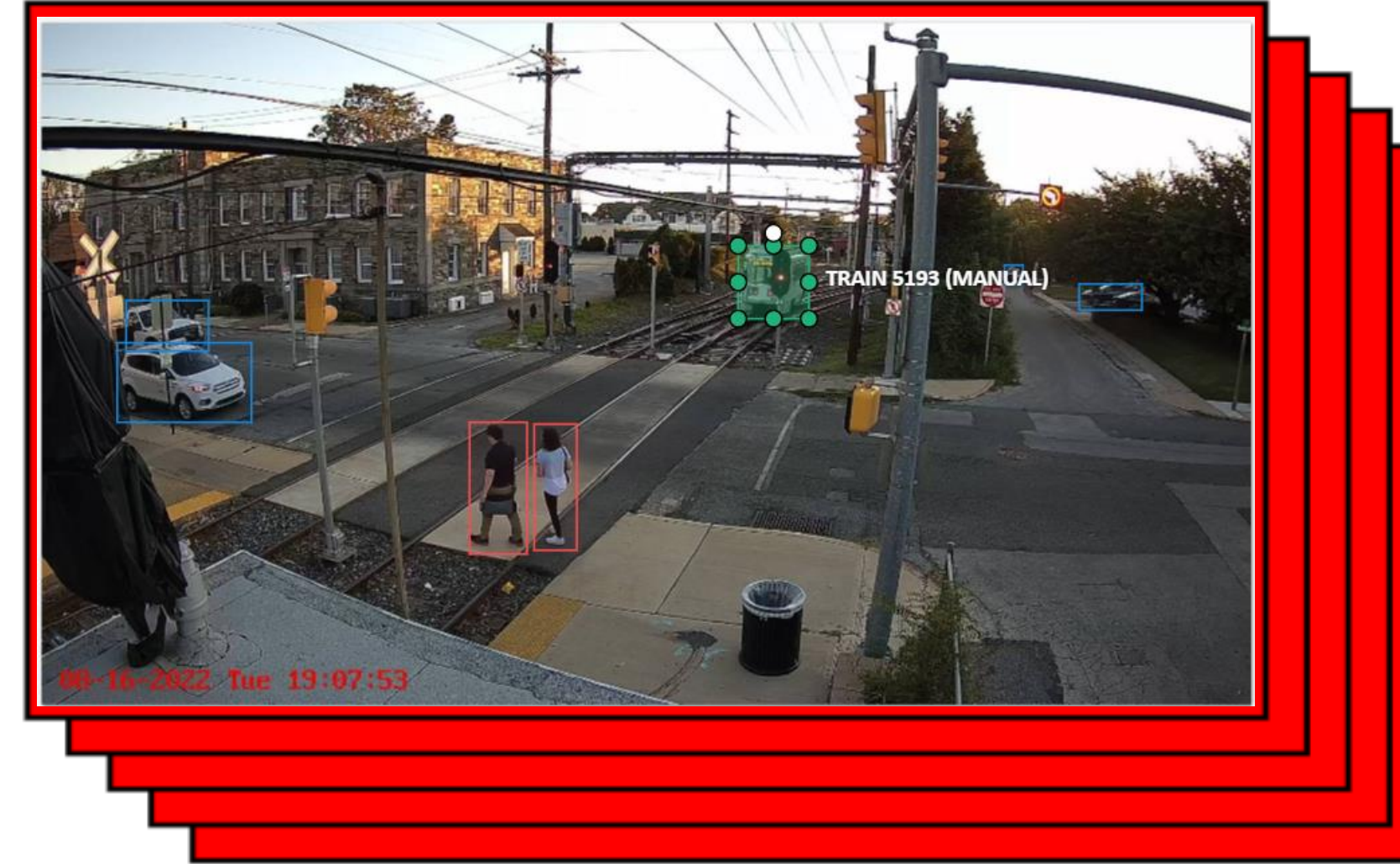
Current method of a barrier + red light is not entirely effective in preventing accidents/trespassing

- Sometimes, people will try to go around the barrier

Object detection has been leveraged for advances in autonomous driving

Materials and Methods

1. Gather + Label Data



Annotated images for object detection

Labels: pedestrian, train, family vehicle

1000's of images → training data, test data

2. Generate Augmented Data Set

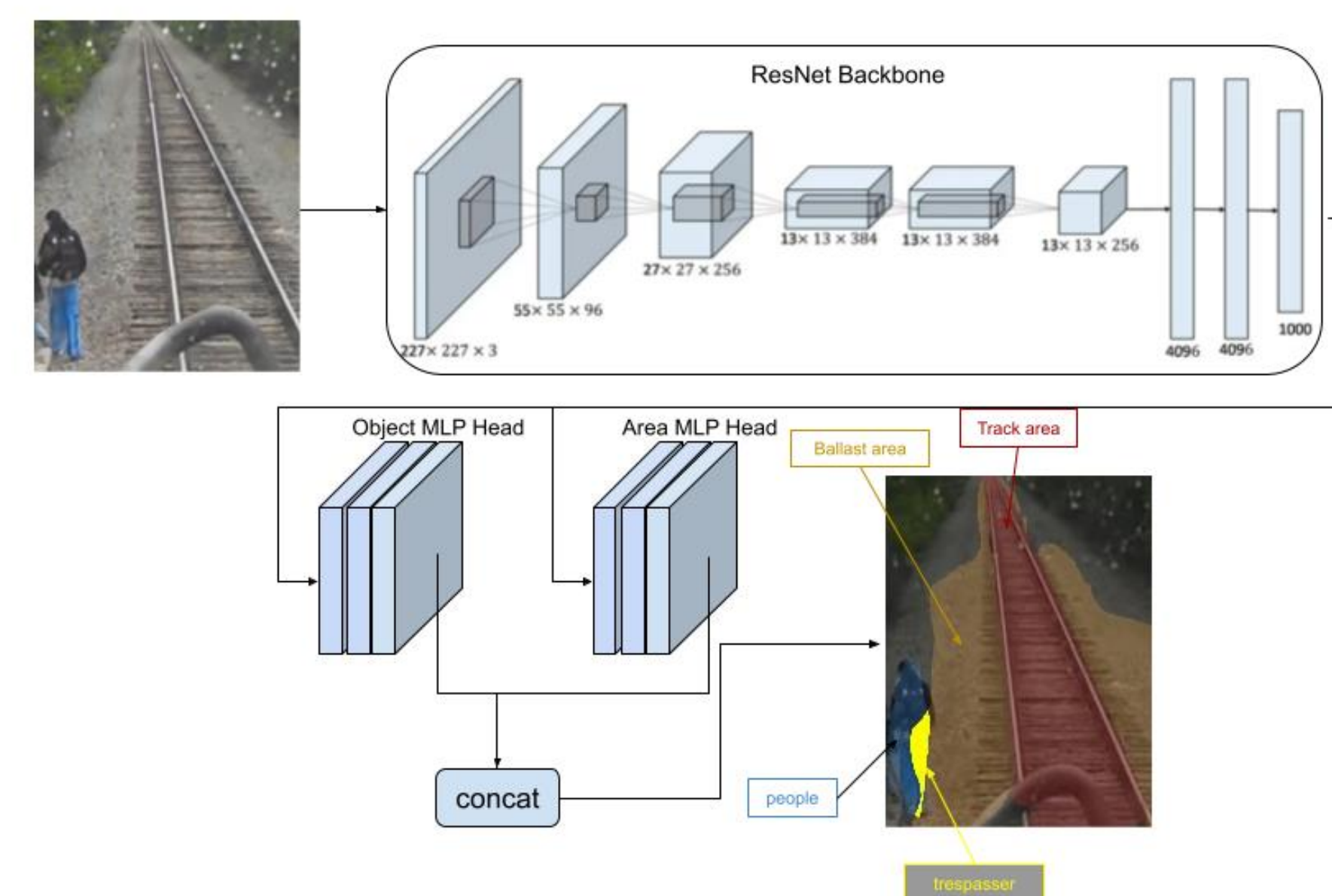


Parse through Microsoft COCO dataset - images of people

- extract a mask of the person
- overlay onto image of railway tracks

Motivation: in our data set, we do not have a lot of images of pedestrians while they are trespassing

3. Build and Run the Machine Learning Model



→ Iterate through the training data

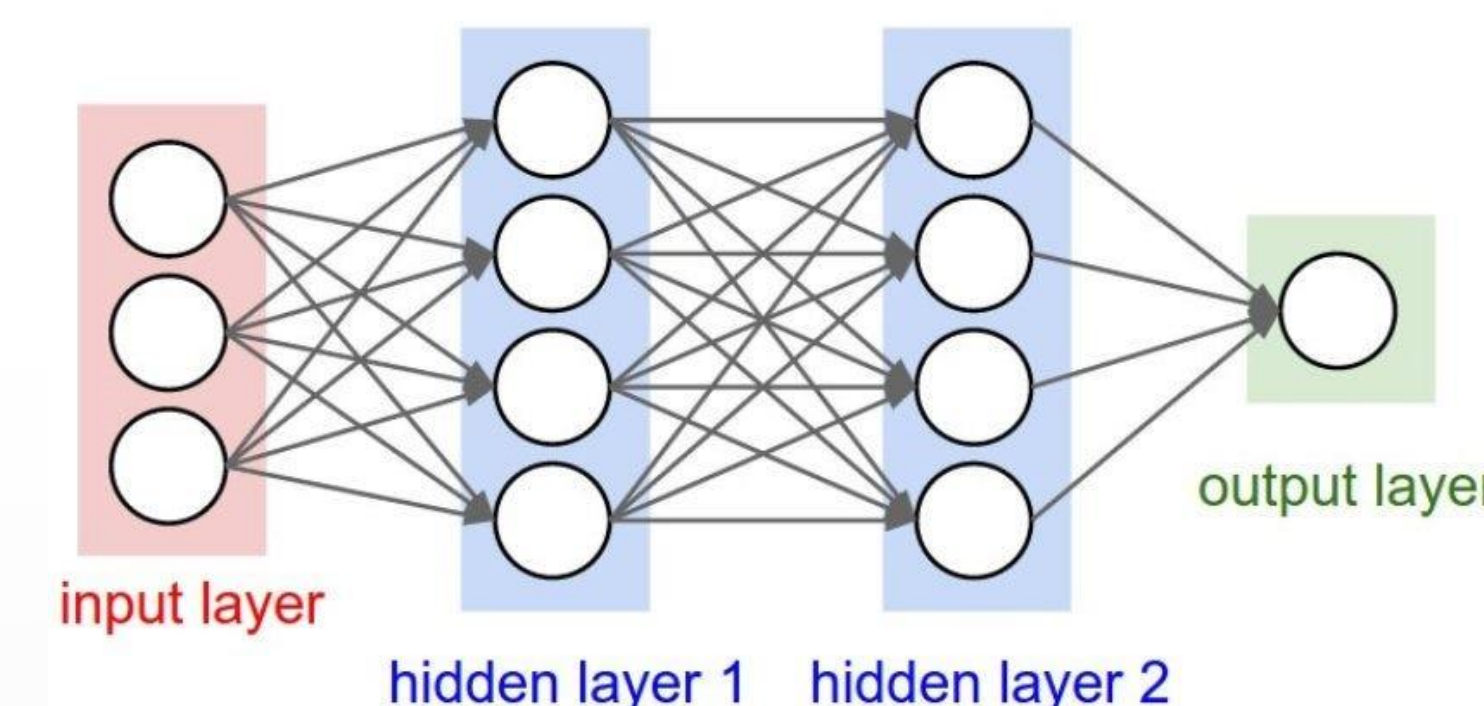
→ Identify patterns and features

→ Make predictions for test data

4. Improve the Model

Fine-tune **hyperparameters**:

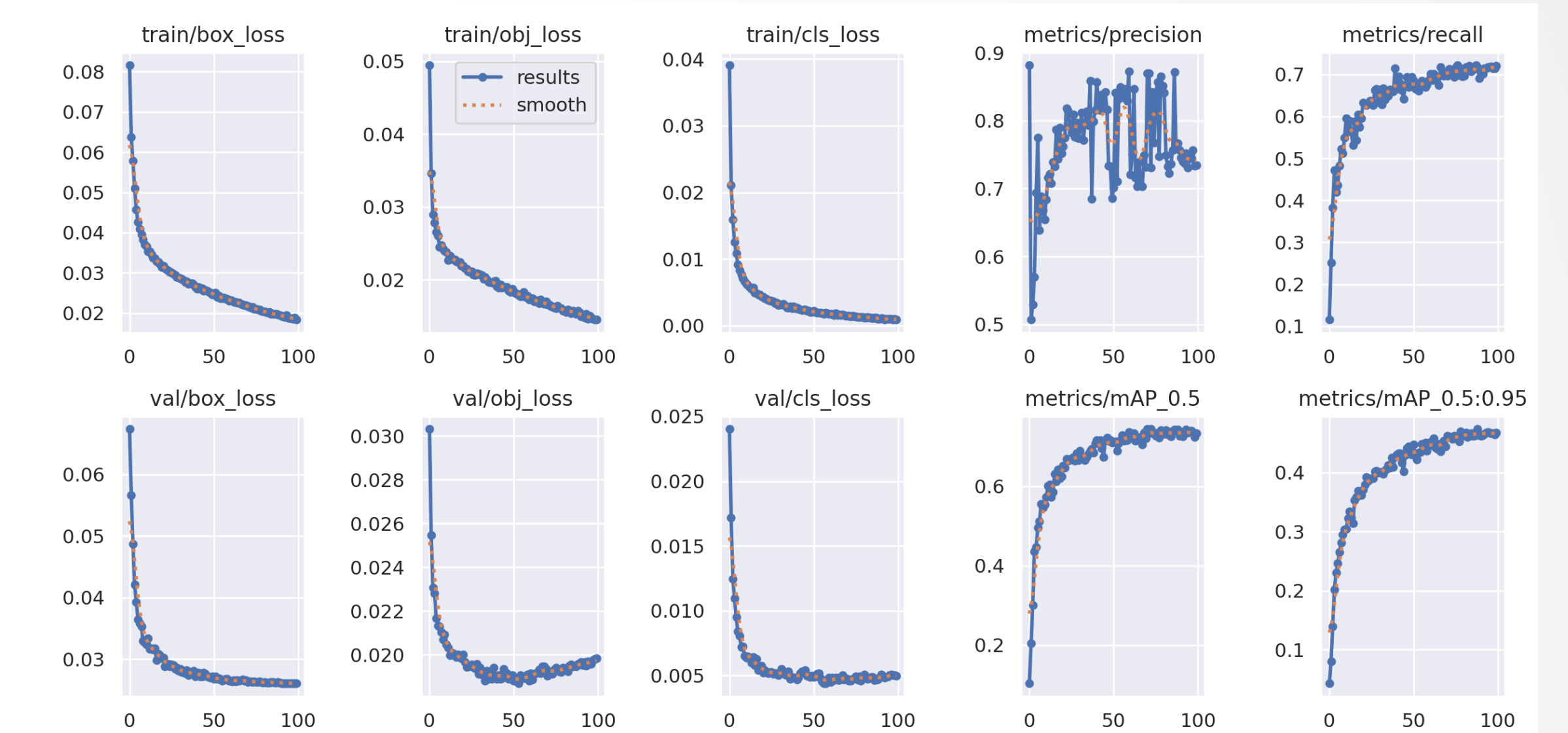
- Learning rate
- Number of layers
- Number of epochs (passes through the entire dataset)



Results



Artificially generated trespassing incident



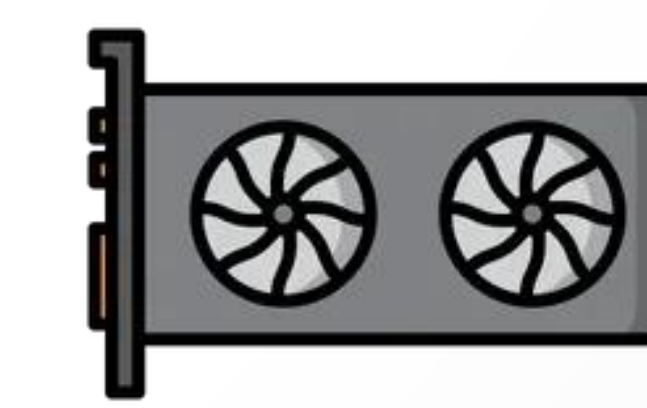
Sample Training and Test Loss Data

- Is the model **overfitting**? It can do well on the training data set, but cannot extrapolate to the test data set

Conclusion

Computer vision can be applied to solve real world problems by leveraging a computer's resources to draw insights from image data. As developers working to improve a computer vision model, robust data that captures a variety of test cases is crucial for the model to make accurate predictions in a confident manner. When training a computer vision model, it is also important to consider the trade-offs between time, computing resources, and accuracy.

Future Direction



- Leverage newer, more advanced hardware to train models more efficiently
- Dashboard that monitors real-time trespassing events
- Gather data to identify trespassing patterns and trends – days of week, times of day

Acknowledgments

Thanks to Professor Xiang Liu and Asim Zaman for the opportunity to work in this lab and learn through practical experiences. Special thanks to Huixiong for his consistent mentorship and helpful advice. It has been a pleasure working with him. I have learned a lot about the field of computer science from him, and I thank him for furthering my interest in computer vision.